

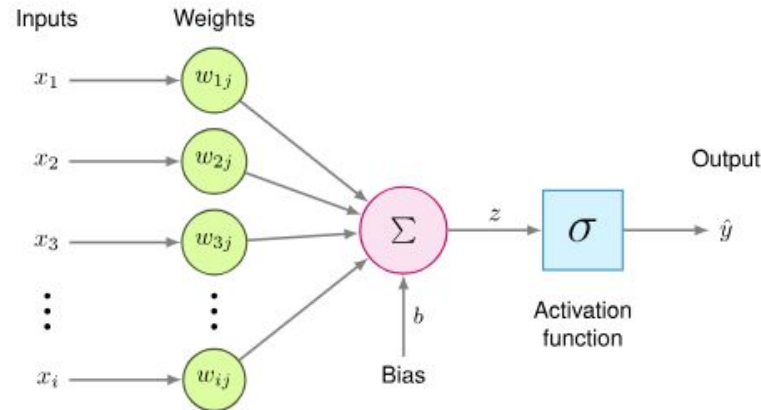
Overview of Deep Learning Concepts

Mythili Vutukuru

Credits: <https://www.mlsysbook.ai/>

<https://thelmlbook.com/>

The perceptron



$$z = \sum (x_i \cdot w_{ij}) + b$$

$$\hat{y} = \sigma(z)$$

$$\hat{y} = \sigma \left(\sum (x_i \cdot w_{ij}) + b \right)$$

Figure 10: **Weighted Input Summation:** Perceptrons compute a weighted sum of multiple inputs, representing feature values, and pass the result to an activation function to produce an output. each input x_i is multiplied by a corresponding weight w_{ij} before being aggregated, forming the basis for learning complex patterns from data.

A simple neural network “layer”

What is more
efficient? $X^T W$
or $W^T X$

Consider a layer with “n” inputs and “m” neurons

A single neuron “j” computes its output as $z = \sum_{i=1}^n (x_i \cdot w_{ij}) + b.$

This calculation is repeated at all “m” neurons with different weights

Consider a **weights matrix** W with dimension $n \times m$ – weight $w(i,j)$ is strength of connection between input feature “i” and neuron “j”

We can compute all “m” outputs using **matrix multiplication** $\mathbf{z} = \mathbf{x}^T \mathbf{W} + \mathbf{b}$

Input X is a vector, or matrix if computing for batch of inputs

Matmul is the most important building block of all ML operations

A “feedforward” neural network

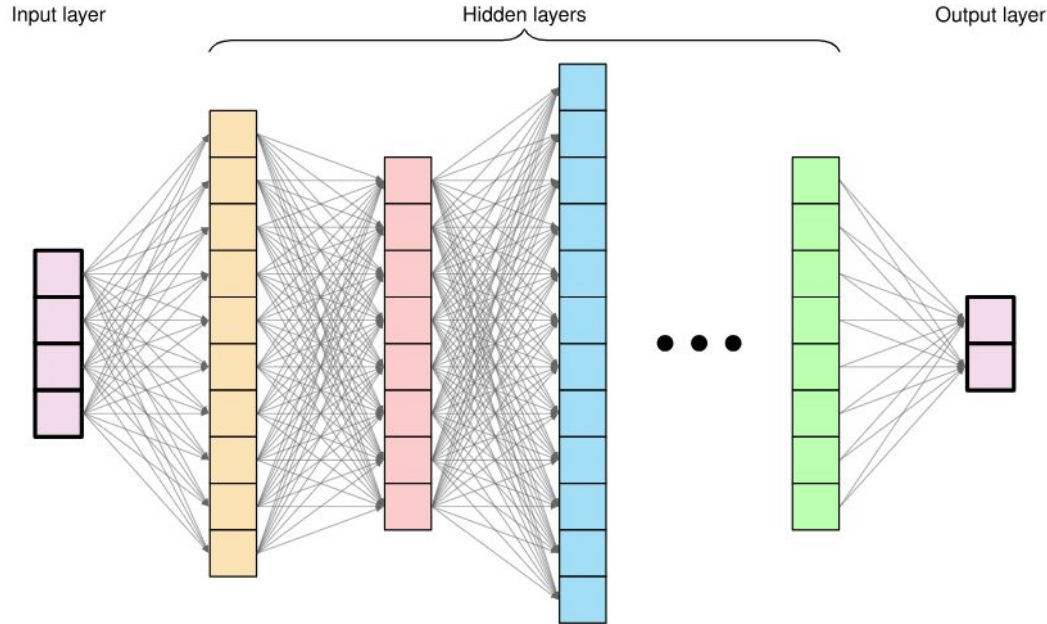
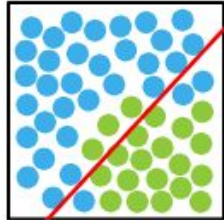


Figure 13: **Layered Network Architecture:** Deep neural networks transform data through successive layers, enabling the extraction of increasingly complex features and patterns. Each layer applies non-linear transformations to the outputs of the previous layer, ultimately mapping raw inputs to desired outputs.

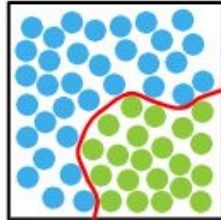
Source: brunellon.

Non-linear activation functions

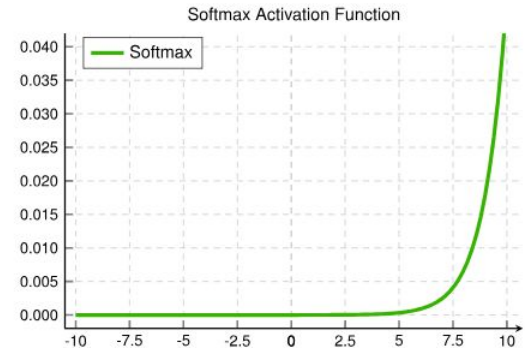
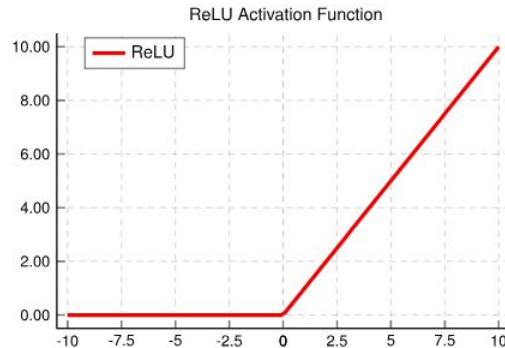
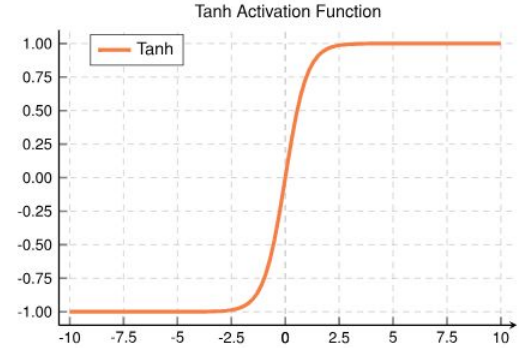
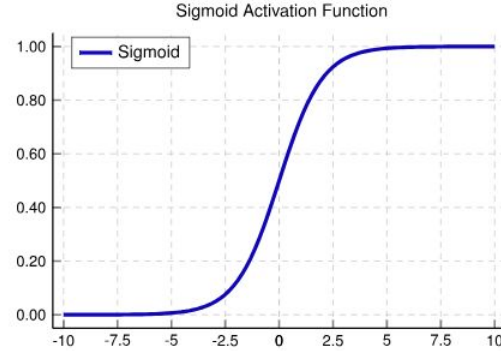
Why non-linear? Allows to combine multiple layers (else, all collapse into one linear transformation)



Neural Network without an Activation Function



Neural Network with an Activation Function



Example: a simple neural network to recognize digits

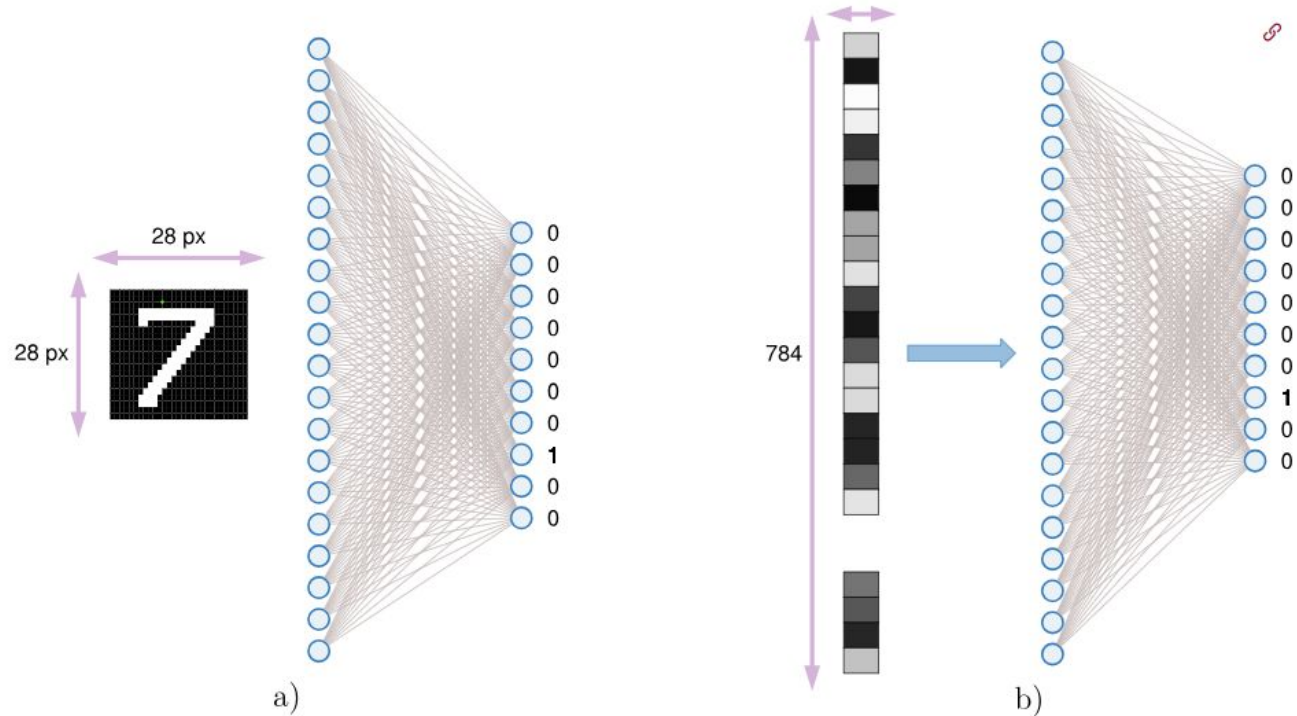
MNIST digit
classification

$28 \times 28 = 784$ inputs

Two hidden layer of 10
neurons each

Output layer of 10
neurons, correspondir
to digits

Total parameters =
 $784 \times 100 + 100 \times 100 +$
 $100 \times 10 + \text{biases}$



Depth vs width vs performance

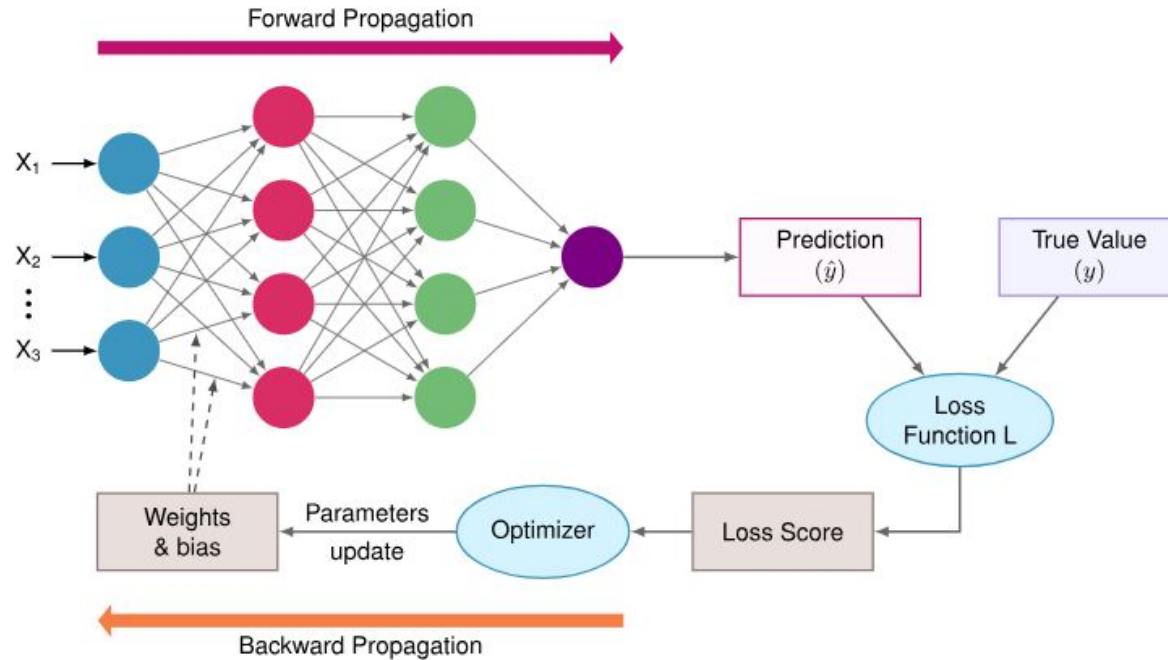
Depth of network (number of layers) determines the complexity of patterns it can capture, but deeper networks are also harder to train

Width of layer (number of neurons in each layer) determines how much data can be handled in parallel, how many different patterns can be detected

Depth and width together determine parameter count – adjacent layers of size “ n_1 ” and “ n_2 ” require $n_1 \times n_2$ weights, plus layer of n neurons needs n bias terms

Depth and width determine compute and memory needs, amount of training data needed to learn all parameters, training convergence time, inference accuracy etc.

Training



Different loss functions used for different ML tasks

Loss computed over batch of inputs

Gradient of loss wrt weights is used to update the weights (in proportion to a learning parameter)

$$\theta_{\text{new}} = \theta_{\text{old}} - \alpha \nabla_{\theta} L$$

Forward pass

For a network with L layers, we can express the full forward computation as:

$$\mathbf{A}^{(L)} = f^{(L)}\left(\mathbf{W}^{(L)} f^{(L-1)}\left(\mathbf{W}^{(L-1)} \dots \left(f^{(1)}(\mathbf{W}^{(1)} \mathbf{X} + \mathbf{b}^{(1)})\right) \dots + \mathbf{b}^{(L-1)}\right) + \mathbf{b}^{(L)}\right)$$

While this nested expression captures the complete process, we typically compute it step by step:

1. First layer:

$$\mathbf{Z}^{(1)} = \mathbf{W}^{(1)} \mathbf{X} + \mathbf{b}^{(1)}$$

$$\mathbf{A}^{(1)} = f^{(1)}(\mathbf{Z}^{(1)})$$

2. Hidden layers ($l = 2, \dots, L - 1$):

$$\mathbf{Z}^{(l)} = \mathbf{W}^{(l)} \mathbf{A}^{(l-1)} + \mathbf{b}^{(l)}$$

$$\mathbf{A}^{(l)} = f^{(l)}(\mathbf{Z}^{(l)})$$

3. Output layer:

$$\mathbf{Z}^{(L)} = \mathbf{W}^{(L)} \mathbf{A}^{(L-1)} + \mathbf{b}^{(L)}$$

$$\mathbf{A}^{(L)} = f^{(L)}(\mathbf{Z}^{(L)})$$

Loss function

Loss function computed by comparing output of final layer with expected output

$$L(\hat{y}, y) = \text{measure of discrepancy between prediction and truth}$$

Many functions possible, e.g., root mean squared error, cross entropy loss

Cross entropy loss function for digits classification

$$L(\hat{y}, y) = - \sum_{j=1}^{10} y_j \log(\hat{y}_j)$$

Loss is 0 if output is 1 for correct digit and 0 for wrong digits, non-zero otherwise

Loss can be averaged across batch of training samples

Backward pass gradient descent

Compute gradient of loss wrt parameters, use this to adjust parameters

Compute partial derivative of loss function with respect to parameters (chain rule)

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial A} \frac{\partial A}{\partial Z} \frac{\partial Z}{\partial W}$$

Diagram illustrating the chain rule for the backward pass gradient descent:

- $\frac{\partial L}{\partial W}$: Gradient used for weight update for each layer
- $\frac{\partial L}{\partial A}$: Computed from error term in last layer, and from propagating error backwards in other layers
- $\frac{\partial A}{\partial Z}$: Differential of activation function $f'(z)$
- $\frac{\partial Z}{\partial W}$: Equal to inputs or activation of prev layer (from $\mathbf{Z} = \mathbf{XW} + \mathbf{b}$)

Gradients computation

At each layer l , we compute three main gradient components:

1. Weight Gradients:

$$\frac{\partial L}{\partial \mathbf{W}^{(l)}} = \frac{\partial L}{\partial \mathbf{Z}^{(l)}} \mathbf{A}^{(l-1)T}$$

because $\frac{\partial Z^{(l)}}{\partial W^{(l)}} = A^{(l-1)}$

2. Bias Gradients:

$$\frac{\partial L}{\partial \mathbf{b}^{(l)}} = \frac{\partial L}{\partial \mathbf{Z}^{(l)}}$$

because $\frac{\partial Z}{\partial b} = 1$

3. Input Gradients (for propagating to previous layer):

$$\frac{\partial L}{\partial \mathbf{A}^{(l-1)}} = \mathbf{W}^{(l)T} \frac{\partial L}{\partial \mathbf{Z}^{(l)}}$$

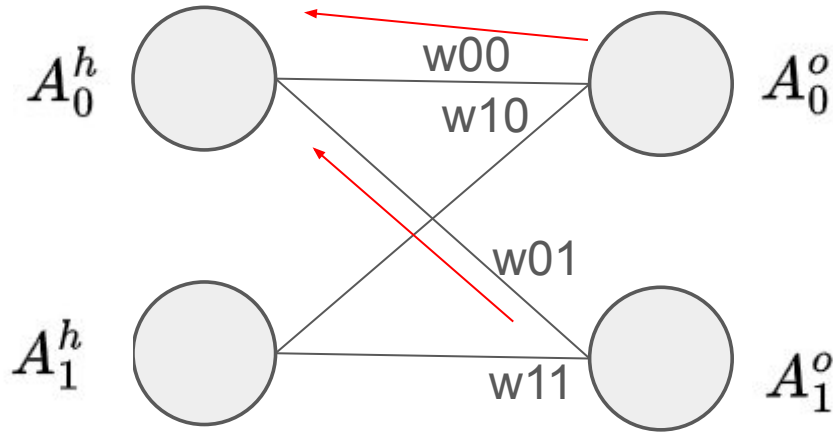
because $\frac{\partial Z^{(l)}}{\partial A^{(l-1)}} = W^{(l)}$

From error or step 3

$\frac{\partial L}{\partial A} \frac{\partial A}{\partial Z}$ $\rightarrow$ $\sigma'(Z)$

Illustration of step 3

$$\begin{bmatrix} A_0^h & A_1^h \end{bmatrix} \begin{bmatrix} w_{00} & w_{01} \\ w_{10} & w_{11} \end{bmatrix} = \begin{bmatrix} Z_0^o & Z_1^o \end{bmatrix}$$



$$\frac{\partial L}{\partial A_0^h} = w_{00} \frac{\partial L}{\partial Z_0^o} + w_{01} \frac{\partial L}{\partial Z_1^o}$$

Gradients of a layer reused to
compute gradients of previous layer

Summary of the training process

1

Run **forward pass** through all layers (matmul of weights and prev layer activations)

- Store intermediate layer outputs (Z and A)

Compute final **loss** by comparing to expected output

- Use this final loss to compute weight and bias gradients at final layer

Propagate loss gradient backwards to previous layers (**back propagation** of loss)

- Backward pass takes longer (must compute weight gradient and input gradient)

Run an **optimizer** to update weights of all layers using the gradients

Gradients can be averaged over multiple samples in a **batch** before weight update

- Batching allows multiple inputs to be processed together in one matmul

- But requires activations of entire batch to be in memory till optimizer runs

Resource requirements for training

Compute requirements: each layer requires matrix multiplication (twice as much in backward pass as compared to forward pass, for weight and input gradients), vector addition of bias, non-linear activation function calculation

Memory requirements: model parameters (weights and bias terms), intermediate activations (for gradient computations), gradients (for final weight updates)

you may also have to store gradients

Some optimizers maintain additional states (e.g., momentum) for gradient descent

GPUs better optimized for high parallel matrix multiplication operations, and faster memory access during computations, but single GPU may not suffice

Training large models requires **multiple GPUs** for long periods of time

Choice of activation functions

Sigmoid and tanh suffer from **vanishing gradient problem** (gradient becomes small as loss propagates, slows down learning)

ReLU is easy to compute, no vanishing gradient, revolutionized deep learning

Softmax is useful to map activation into probability distribution across classes

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

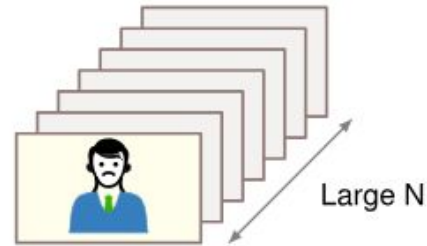
$$\text{ReLU}(x) = \max(0, x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$$

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

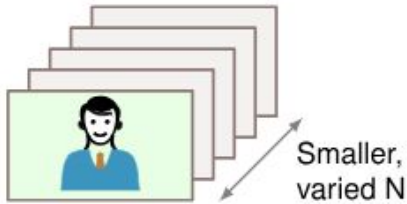
Inference

Computationally cheaper than training, as only forward pass, and fewer inputs

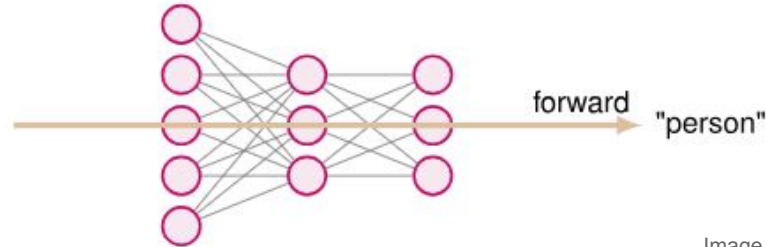
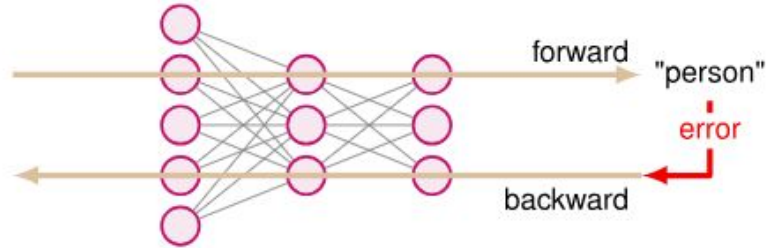
Lesser memory as no need to store intermediate activations for backward pass



Training



Inference



Deep learning architectures

Multi-layer perceptron (MLP), the simplest “deep” neural network, set of several fully-connected layers – can capture complex patterns with enough depth

Convolutional neural networks (CNN) for visual tasks

Recurrent neural networks (RNN) for language tasks

Transformers is the architecture underpinning LLMs

We will aim to understand each DNN architecture in terms of its compute, memory, and data movement requirements. Later, we will understand how these requirements are realized in hardware and software.

MLP for MNIST digit classification network

784 inputs, 2 hidden layers of 100 neurons each, 10 neurons in output

Number of weights = $78,400 + 10,000 + 1000$ (and as many gradients)

Bias terms = $100 + 100 + 10$ (and as many activations)

In the first layer, each neuron performs 784 multiply-accumulate operations, by accessing 784 inputs and 784 weights, to compute one output value (repeated across all 100 neurons) = 78400 multiply-adds

Multiplication of 100×784 weight matrix with 784×1 input vector

Note: multiplication of $M \times K$ and $K \times N$ matrices into $M \times N$ product needs

$O(MNK)$ compute and $O(MN + NK + MK)$ storage

MLP: Systems implications

Large number of parameters due to dense connections between layers

High memory usage for parameters, activations, gradients

High compute intensity (large matmuls), but easily parallelizable via GPUs

Not enough data locality – every output needs every input and corresponding weights – needs large data movement between compute and memory

Training is more intensive – backprop needs twice as much compute as forward pass, and a lot more memory to store activations

Convolutional Neural Networks

CNNs are better suited for spatial pattern processing, where there is some locality and spatial pattern in the data

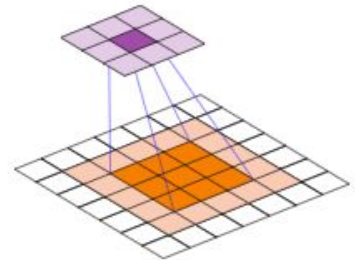
Filter or kernel – a small $d_i \times d_j$ dimension weight matrix that extracts some feature

In a convolution layer, a kernel runs over all pixels and computes a convolution of kernel weights with inputs to the layer, i.e., local weighted sum

$$H_{i,j}^{(l)} = \sum_{d_i} \sum_{d_j} W_{d_i,d_j}^{(l)} H_{i+d_i,j+d_j}^{(l-1)} + b$$

Further, non-linear ReLU before output

Process repeated for input and output channels, e.g., RGB values



Algorithmic structure of CNNs

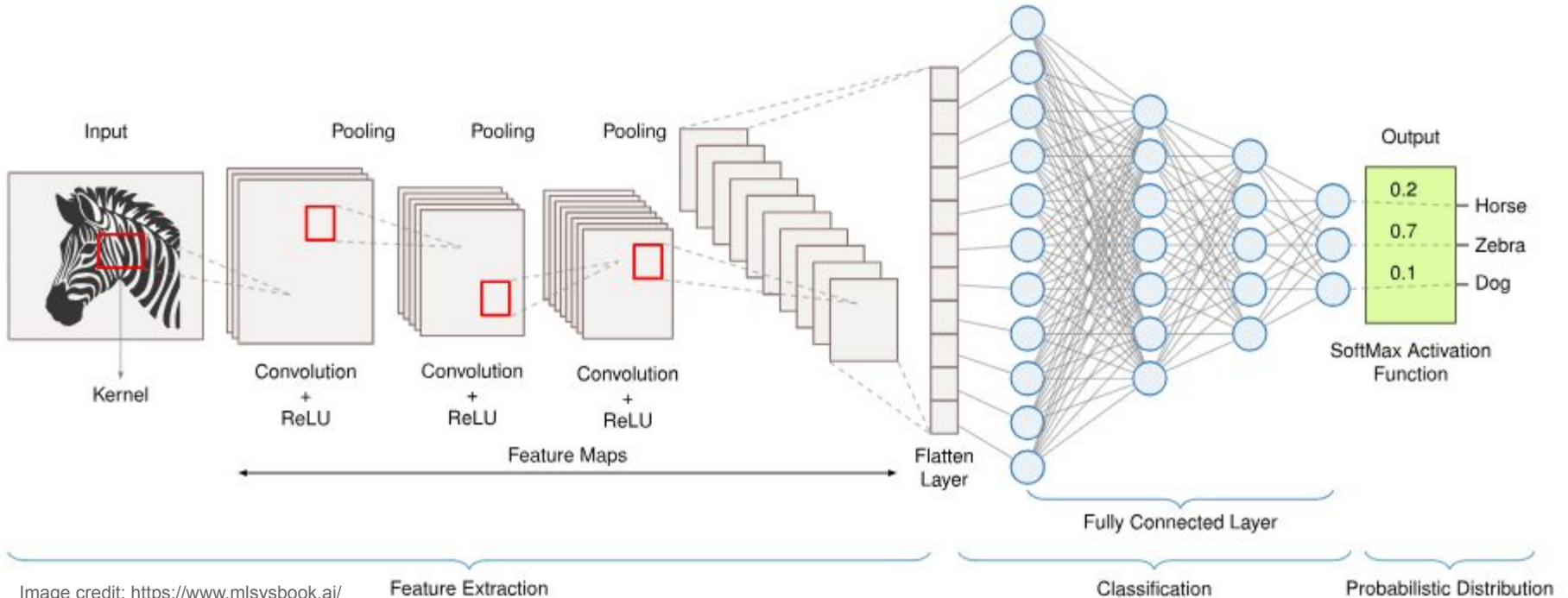
The core operation in a CNN can be expressed mathematically as:

$$\mathbf{H}_{i,j,k}^{(l)} = f \left(\sum_{di} \sum_{dj} \sum_c \mathbf{W}_{di,dj,c,k}^{(l)} \mathbf{H}_{i+di,j+dj,c}^{(l-1)} + \mathbf{b}_k^{(l)} \right)$$

This equation describes how CNNs process spatial data. $\mathbf{H}_{i,j,k}^{(l)}$ is the output at spatial position (i, j) in channel k of layer l . The triple sum iterates over the filter dimensions: (di, dj) scans the spatial filter size, and c covers input channels. $\mathbf{W}_{di,dj,c,k}^{(l)}$ represents the filter weights, capturing local spatial patterns. Unlike MLPs that connect all inputs to outputs, CNNs only connect local spatial neighborhoods.

A complete CNN

Conv layers extract features (pooling layers summarize larger layers, reduce size) and these feature maps are fed into fully-connected layers



CNN for MNIST digit classification

A convolutional layer with 28×28 grayscale images inputs

32 filters of size 3×3 , so number of parameters = 288

Output is 32 feature maps of size 28×28 , so each output produces 32 different feature representations of its neighborhood

$28 \times 28 \times 32$ activation values produced for each convolutional layer

Each output requires only 9 multiply-accumulate operations, and needs only a small amount of data (local area of input and small weight matrix) to compute

CNN: Systems implications

Fewer number of parameters compared to MLP, as capturing local features

But large memory requirement to store activations (feature maps)

Smaller amount of compute for each output, with lesser data accessed

Significant spatial locality in the data accessed for compute (filter weights reused, feature map streamed sequentially), so caching is helpful

Recurrent Neural Networks

Language processing tasks require memory or context from previous inputs to better understand the current input, e.g., the meaning of the word “mole” depends on the other words that came before it

RNNs address this issue by maintaining some hidden state across time, so that the output of current time depends not only on current input and weights, but also on activations of the previous time step (connected by recurrent weights)

$$\mathbf{h}_t = f(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h)$$

where $\mathbf{h}_t$ denotes the hidden state at time t , $\mathbf{x}_t$ denotes the input at time t , $\mathbf{W}_{hh}$ contains the recurrent weights, and $\mathbf{W}_{xh}$ contains the input weights, as illustrated in the unfolded network

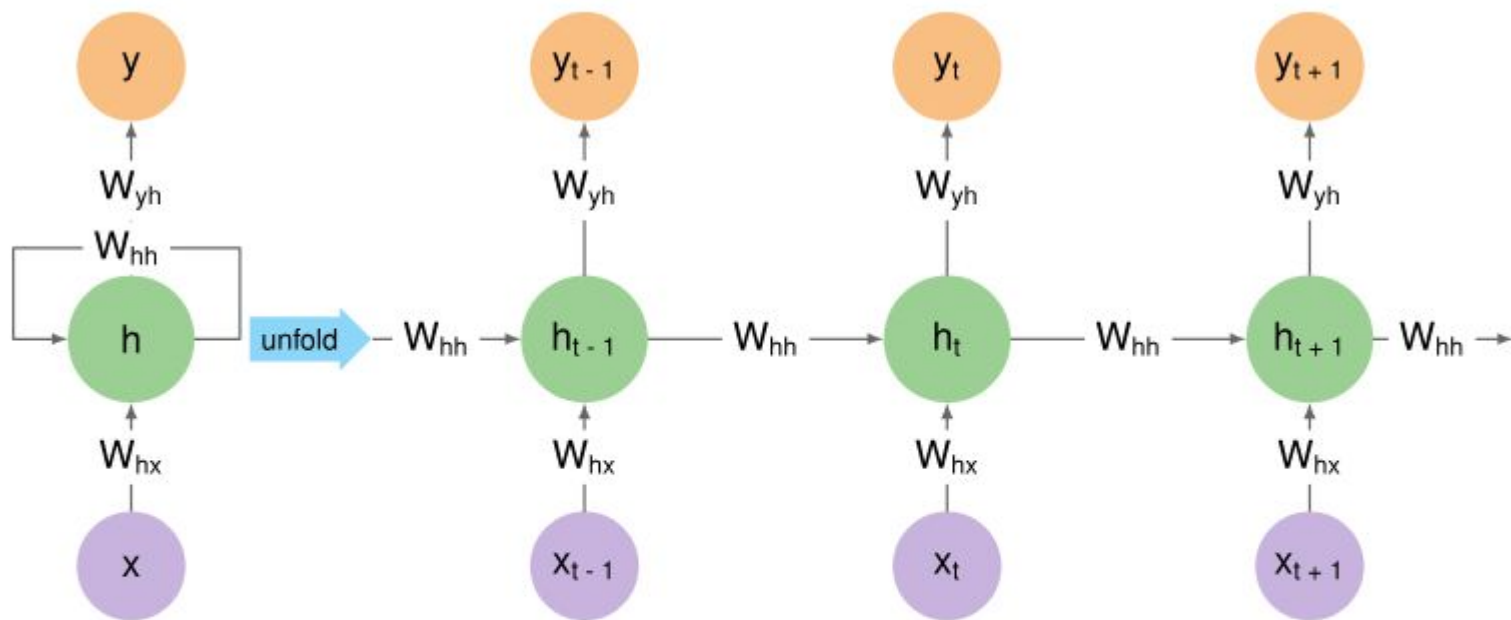


Figure 4: Recurrent Neural Network Unfolding: Rnns process sequential data by maintaining a hidden state that incorporates information from previous time steps through this diagram. the unfolded structure explicitly represents the temporal dependencies modeled by the recurrent weights, enabling the network to learn patterns across variable-length sequences.

Example of RNN

Sequence processing task with input dimension 100, hidden dimension 128

Every input in a sequence is converted into a vector / embedding of size 100 and fed into network

12,800 weights for each input in a sequence, 16,384 for recurrent connection (which are reused for all inputs)

Extra memory required to maintain hidden state activations across time

For every element in sequence, load hidden state (128 values), load weights (29,184 values), compute output via two matrix multiplications (100×128 and 128×128 multiply-accumulates) and store hidden state.

Recurrent connection weights are reused

RNN: Systems implications

Recurrent weights can be cached, temporal locality in data access pattern (unlike spatial locality in CNNs)

Hard to process long sequence lengths, as memory needed to store hidden states increases with sequence length

Hard to parallelize computation, as each time step depends on completion of previous time step – parallelization only possible via batching multiple sequences

Large Language Models (LLMs) and Transformers

Transformer architecture underpins model LLMs

Fixes some issues with RNNs (better captures context, allows parallelization)

RNN is encoder-decoder architecture (encoder processes input, summarizes into a context vector, which is used by decoder to generates output one word at a time)

Transformer is a decoder-only architecture. Decoder consumes all input, builds a larger context, and generates output one word (“token”) at a time

Uses a mechanism called “attention” to capture relationships between input tokens

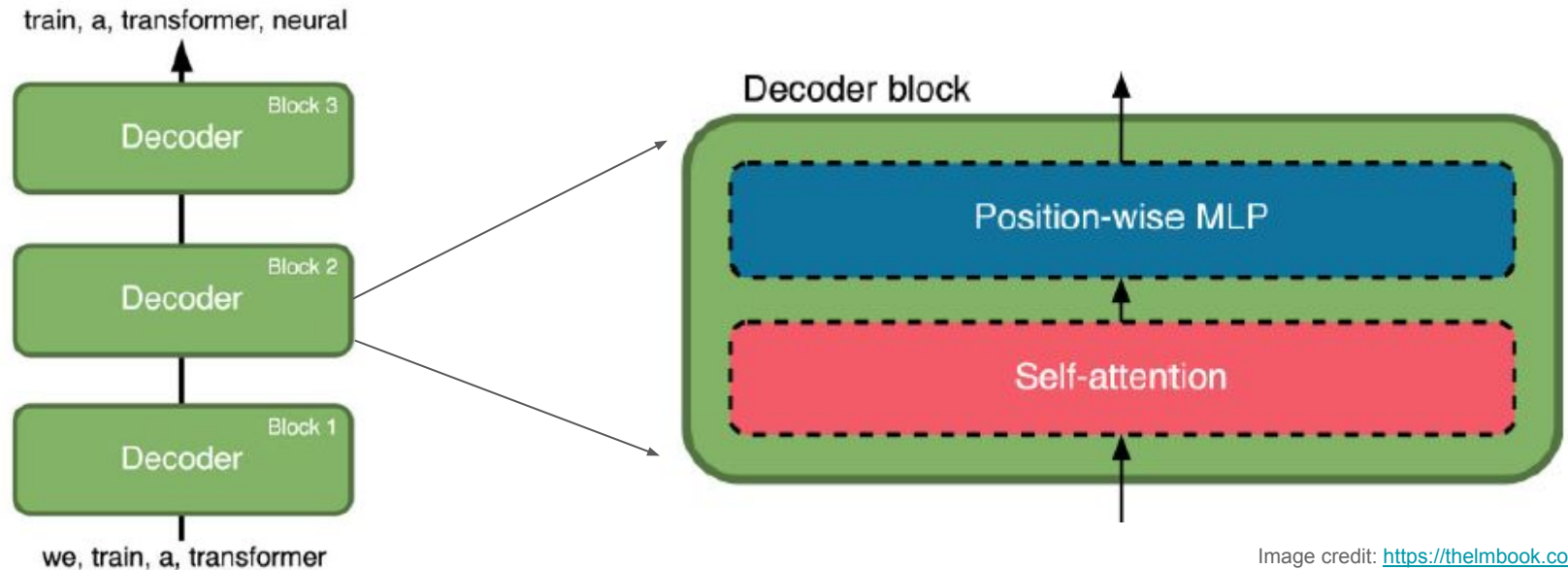
Sequence information between input tokens captured via positional encoding

Watch the “Neural Networks” YouTube playlist by 3Blue1Brown for an excellent introduction to transformers

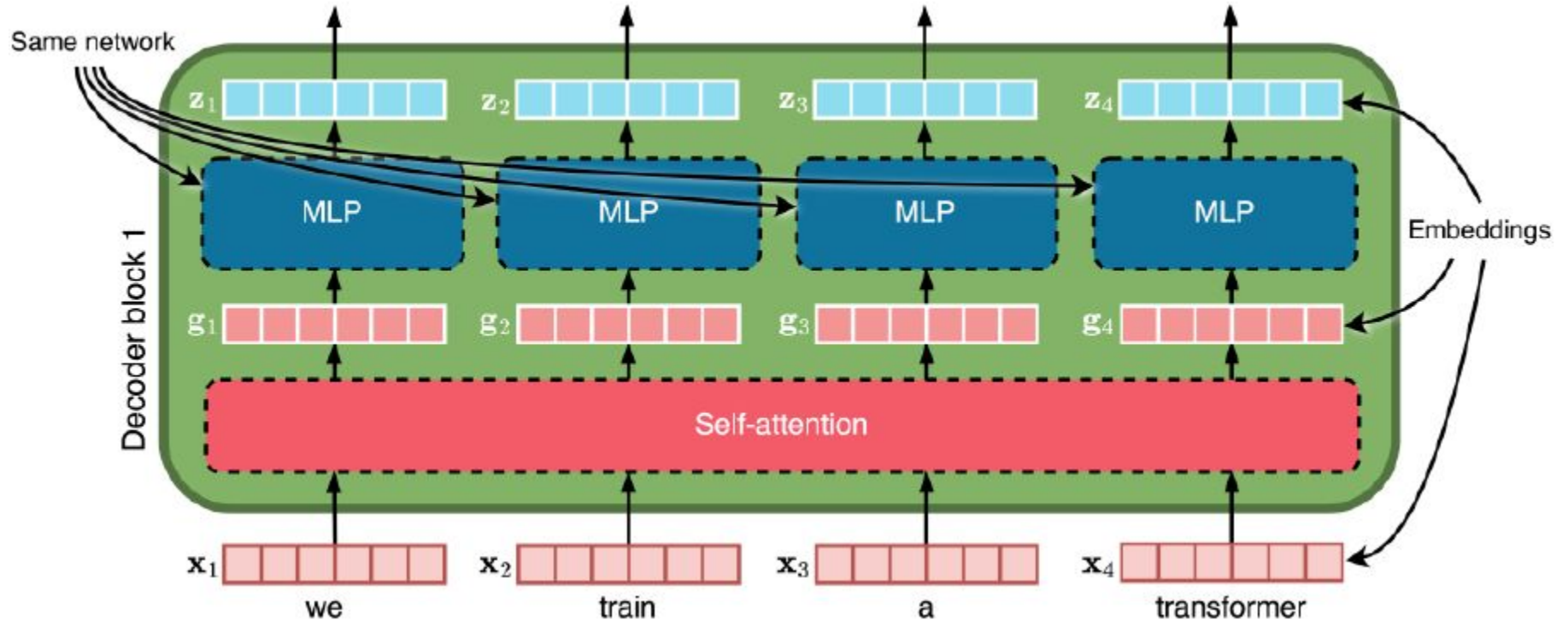
Transformer architecture

Sequence of decoder blocks process input, generate output, token-by-token

Decoder consists of a (mainly) a self-attention layer and MLP layer



Transformer architecture



Token generation using transformers

Most common use case is generating text from input prompt in auto-regressive manner (i.e., a output depends only on its previous inputs)

All processing happens on tokens (like words) converted into embedding vectors

Two phases – prefill and decode

Prefill – all input tokens go through self attention, which changes each input token to capture some context from other tokens before it. This modified input token processed by MLP layer. Attention + MLP repeats many times.

Decode – previous token's output used as input, to generate next output token. Token with highest probability gets picked as generated output (or, we can sample from high prob tokens, search along multiple tokens, ...)

How are embeddings computed? (simplified idea)

Create skipgrams from training text (one word is skipped in a sequence of N words)

Skip-gram	Skipped word
professor alan * research advanced	turing's
alan turing's * advanced computer	research
turing's research * computer science	advanced

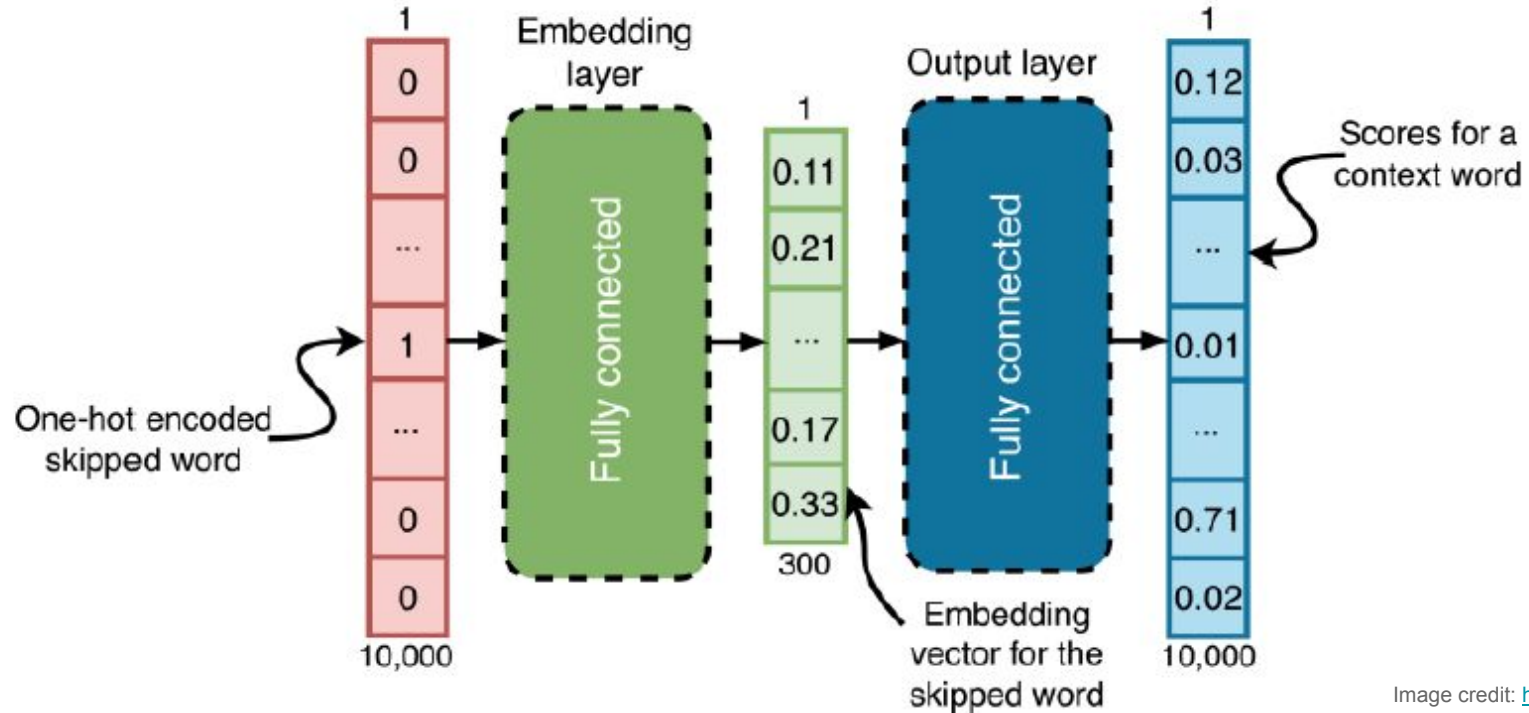
Train network to predict context from skipped word or vice versa

Skipped word (input)	Context word (target)	Position
turing's	professor	-2
turing's	alan	-1
turing's	research	+1
turing's	advanced	+2

Activations in such a network represent the word semantics well

How are embeddings computed?

Train the full network, then use only intermediate layer output as embedding



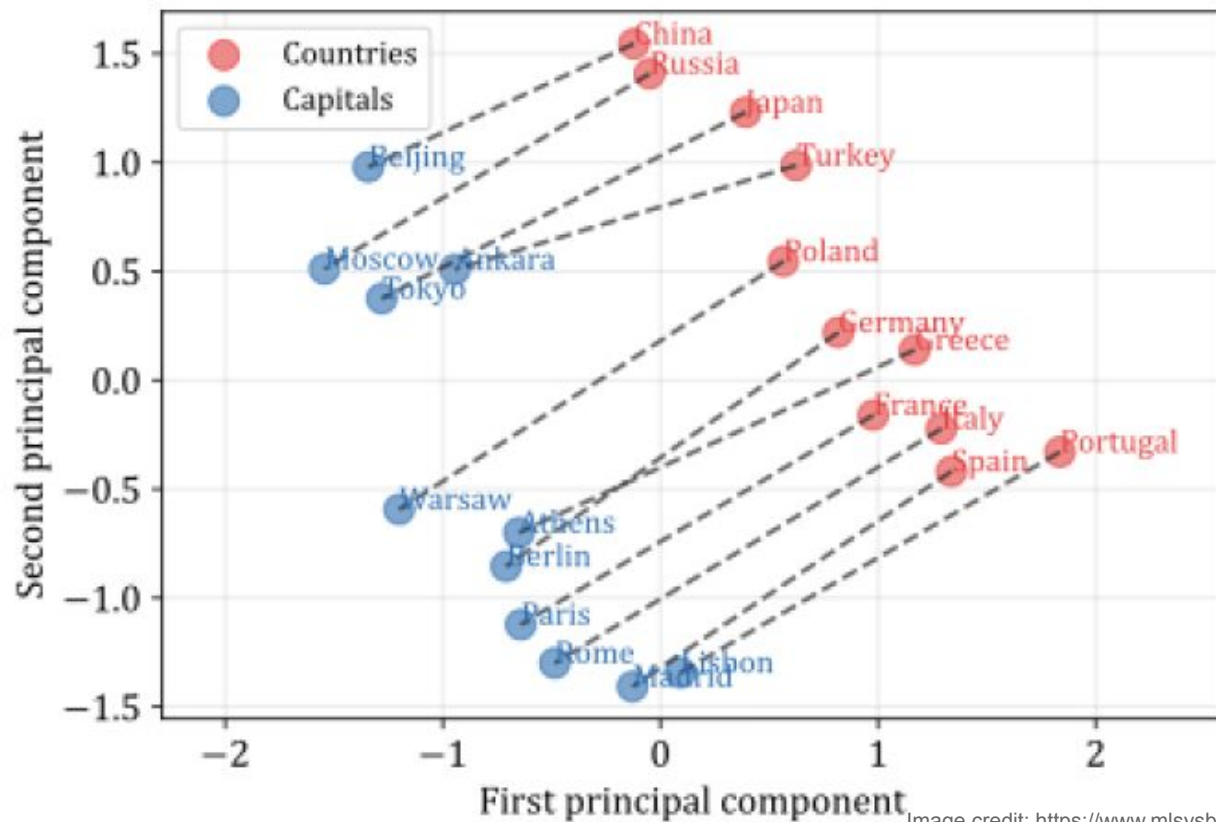
Embeddings encode semantic meaning

For example, we
could have:

$E(\text{king}) - E(\text{queen})$

is almost equal to

$E(\text{man}) - E(\text{woman})$



Self attention

$$\text{SelfAttention}(\mathbf{X}) = \text{softmax} \left(\frac{\mathbf{XW}_Q(\mathbf{XW}_K)^T}{\sqrt{d_k}} \right) \mathbf{XW}_V$$

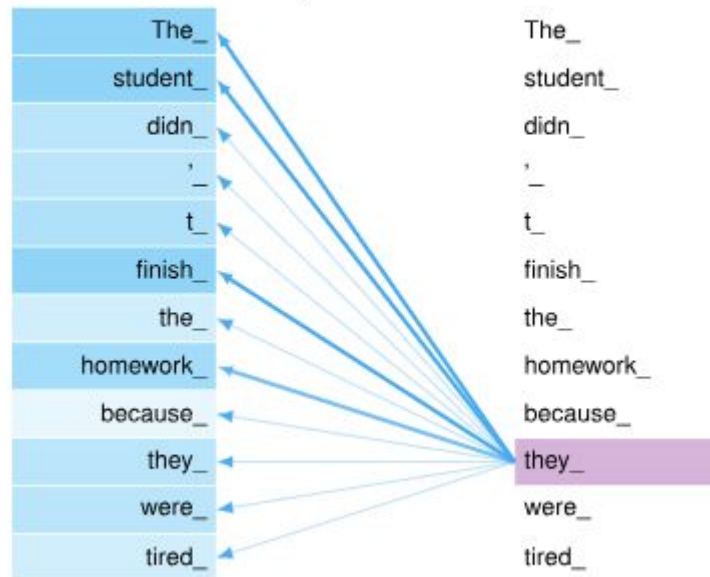
Attention captures relationships and context between words in the input text

Uses 3 weight matrices, called query, key, value to generate an attention score for each input token

Attention score added to input (to capture context), then fed into MLP

The student didnt finish the homework because they were tired.

Layer: 4 Head: 2



Self attention: all the steps in detail

N = sequence length of input

E = embedding dimension

d = model dimension

Input **X** of size N x E

$$\mathbf{G} = \text{attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \stackrel{\text{def}}{=} \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{M}\right) \mathbf{V}$$

1. Multiple X with weights to get Q, K, V
2. Compute Q-K dot product
3. Scale attention values, apply causal mask
4. Compute softmax
5. Multiply with V to get attention score

Attention score added to input, then MLP

Step 1: Generate Q, K, V

X is N x E

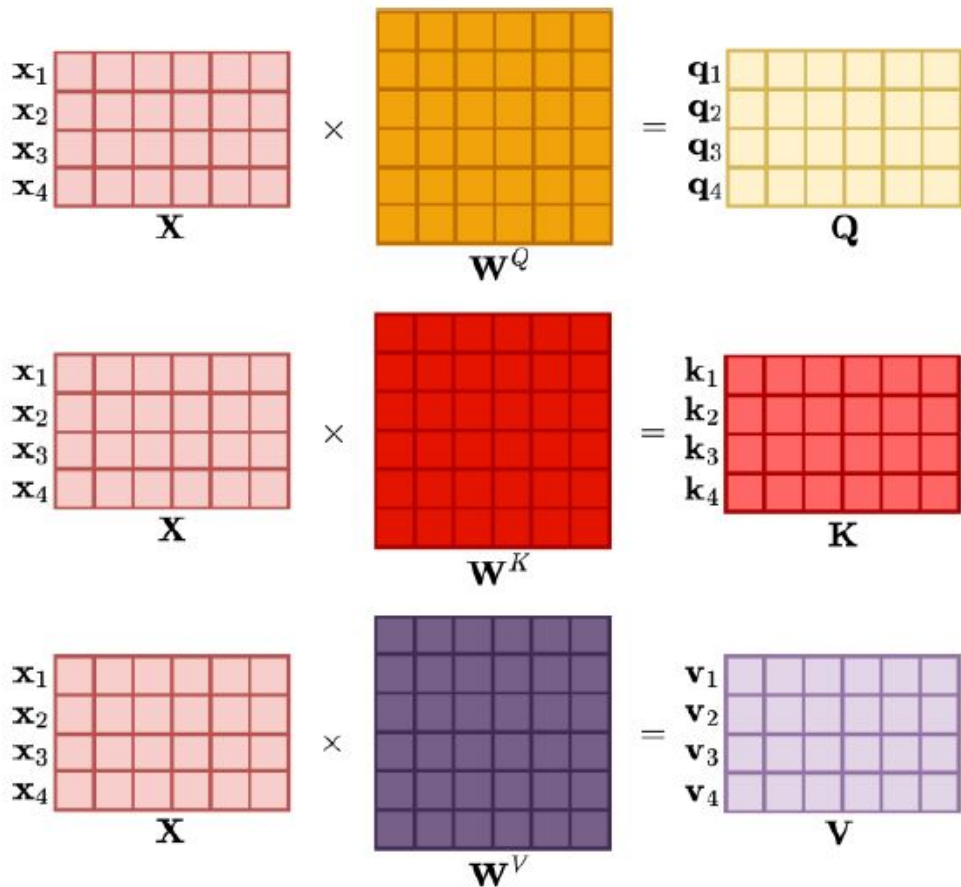
W_Q and W_K are E x d

Q, K computed are N x d

(one d-sized vector for each input)

W_V is E x E (bigger, for now)

V computed is N x E



Step 2: Compute Q-K dot product

Q matrix has N vectors of size d

Similarly, K matrix also

Each Q_i and K_i is vector of size d

Compute dot products between vectors of Q and K as shown

N x N matrix computed

N x N x d multiply-accumulate operations required

	K1	K2	K3	K4
Q1	Q1.K1	Q1.K2	Q1.K3	Q1.K4
Q2	Q2.K1	Q2.K2	Q2.K3	Q2.K4
Q3	Q3.K1	Q3.K2	Q3.K3	Q3.K4
Q4	Q4.K1	Q4.K2	Q4.K3	Q4.K4

What does this mean?

No one knows, but here is a rough idea

Queries ask various questions, looking for information about the word

Keys provide various types of answers or labels

Dot product of queries and keys indicates the correlation, or how well a key answers a query

If dot product is high, then the tokens “attend to” each other, words are related

Step 3: Scale, add causal mask

We want dot products of a row to only depend on tokens before it

We only want dot product of Q2 with K1 and K2, not K3 and K4

Retain only the entries we want, set rest to negative infinity

Why not 0? Because we have softmax coming up next

We also scale each value by $\sqrt{d}$ to avoid values blowing up

	K1	K2	K3	K4
Q1	Q1.K1	-inf	-inf	-inf
Q2	Q2.K1	Q2.K2	-inf	-inf
Q3	Q3.K1	Q3.K2	Q3.K3	-inf
Q4	Q4.K1	Q4.K2	Q4.K3	Q4.K4

Step 4: Apply softmax

We want the attention values of every row to become probabilities that sum up to 1

So we apply a row-wise softmax

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

These probabilities indicate how various previous tokens are related to a token

	K1	K2	K3	K4
Q1	<i>prob</i>	0	0	0
Q2	<i>prob</i>	<i>prob</i>	0	0
Q3	<i>prob</i>	<i>prob</i>	<i>prob</i>	0
Q4	<i>prob</i>	<i>prob</i>	<i>prob</i>	<i>prob</i>

Step 5: Multiply with V

Attention matrix of size $N \times N$, one row per token, probability vector, sums to 1

Multiplied with V matrix of size $N \times E$, N vectors of size E (one per token)

Result is N vectors of size E, where each row is weighted sum of all value vectors, weighted by attention weights of that row (e.g., For example, G2 is weighted sum of all value vectors, weighted by weights in 2nd row of attention)

	K1	K2	K3	K4
Q1	<i>prob</i>	0	0	0
Q2	<i>prob</i>	<i>prob</i>	0	0
Q3	<i>prob</i>	<i>prob</i>	<i>prob</i>	0
Q4	<i>prob</i>	<i>prob</i>	<i>prob</i>	<i>prob</i>

X

V1
V2
V3
V4

=

G1
G2
G3
G4

Note on matrix multiplication

$$\begin{bmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \end{bmatrix} \begin{bmatrix} b_{00} & b_{01} \\ b_{10} & b_{11} \\ b_{20} & b_{21} \end{bmatrix}$$

Columns of A, weighted by elements of B

$$\begin{bmatrix} \begin{pmatrix} a_{00} \\ a_{10} \end{pmatrix} b_{00} & \begin{pmatrix} a_{00} \\ a_{10} \end{pmatrix} b_{01} \\ + & + \\ \begin{pmatrix} a_{01} \\ a_{11} \end{pmatrix} b_{10} & \begin{pmatrix} a_{01} \\ a_{11} \end{pmatrix} b_{11} \\ + & + \\ \begin{pmatrix} a_{02} \\ a_{12} \end{pmatrix} b_{20} & \begin{pmatrix} a_{02} \\ a_{12} \end{pmatrix} b_{21} \end{bmatrix}$$

Rows of B, weighted by elements of A

$$\begin{bmatrix} (b_{00} \ b_{01})a_{00} + (b_{10} \ b_{11})a_{01} + (b_{20} \ b_{21})a_{02} \\ (b_{00} \ b_{01})a_{10} + (b_{10} \ b_{11})a_{11} + (b_{20} \ b_{21})a_{12} \end{bmatrix}$$


$$\begin{bmatrix} a_{00}.b_{00} + a_{01}.b_{10} + a_{02}.b_{20} & a_{00}.b_{01} + a_{01}.b_{11} + a_{02}.b_{21} \\ a_{10}.b_{00} + a_{11}.b_{10} + a_{12}.b_{20} & a_{10}.b_{01} + a_{11}.b_{11} + a_{12}.b_{21} \end{bmatrix}$$

What does this mean?

Value vectors indicate how the embedding of words should be updated to reflect context

Attention matrix indicates strength of connections between words

Words which are related to current word will have high attention scores, and will add some component of their value to the current word

Every word embeds some context from all previous related words

MLP after attention

After self attention, the output goes to an MLP layer

$$\mathbf{z}_t = \mathbf{W}_2 \left(\text{ReLU}(\mathbf{W}_1 \mathbf{g}_t + \mathbf{b}_1) \right) + \mathbf{b}_2$$

Two MLP layers, with RELU in between

Hidden dimension dMLP, usually multiple of model size d

For every token, embedding of size E increases to size dMLP, then ReLU, then comes back to size E, added to original embedding

Multi-headed attention

For the same input sequence, the attention operation performed using multiple W_Q , W_K , W_V weight matrices in separate “heads” in parallel

Using multiple heads allows us to capture diverse patterns in the input

The changes to the input embeddings from all heads is added together to get the final result of attention layer (“residual connection”)

Nuances around value vector

Discussion on value vector so far is slightly incorrect

The W_V matrix is actually not $E \times E$, but combination of two smaller matrices

W_{Vdown} is $E \times d$, W_{Vup} is $d \times E$

Multiply input X ($N \times E$) with W_{Vdown} to generate value vectors $Vdown$ ($N \times d$)

Multiplying $N \times N$ attention by $Vdown$ ($N \times d$) to generate $N \times d$ output vector G

Multiply this $N \times d$ output with W_{Vup} ($d \times E$) to generate final $N \times E$ output

This last step of multiplying with W_{Vup} is combined across multiple heads

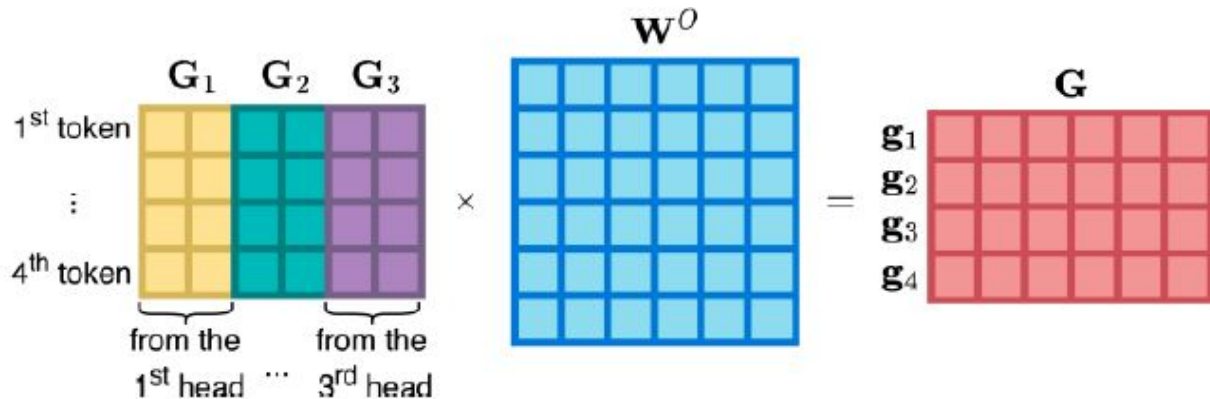
Value vectors with multiple heads

Let there be H heads, each generating $N \times d$ output after multiplying attention scores with value vectors (actually V_{down} vectors)

Concat $N \times d$ outputs of H heads into $N \times E$ matrix (because $d \times H = E$)

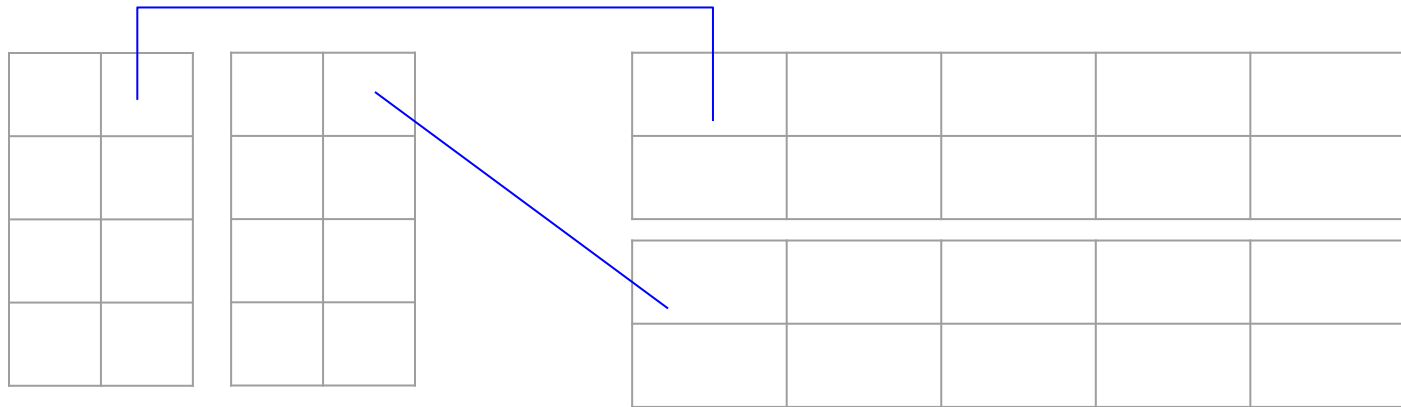
Concat $d \times E$ matrix W_{up} for all heads into output matrix of size $E \times E$

Multiply the above two to get the final output from multi-headed attention



Another view of concatenation and projection

Equivalent to applying V_{down} and V_{up} in each head and adding up the result



$N \times d$ matrices

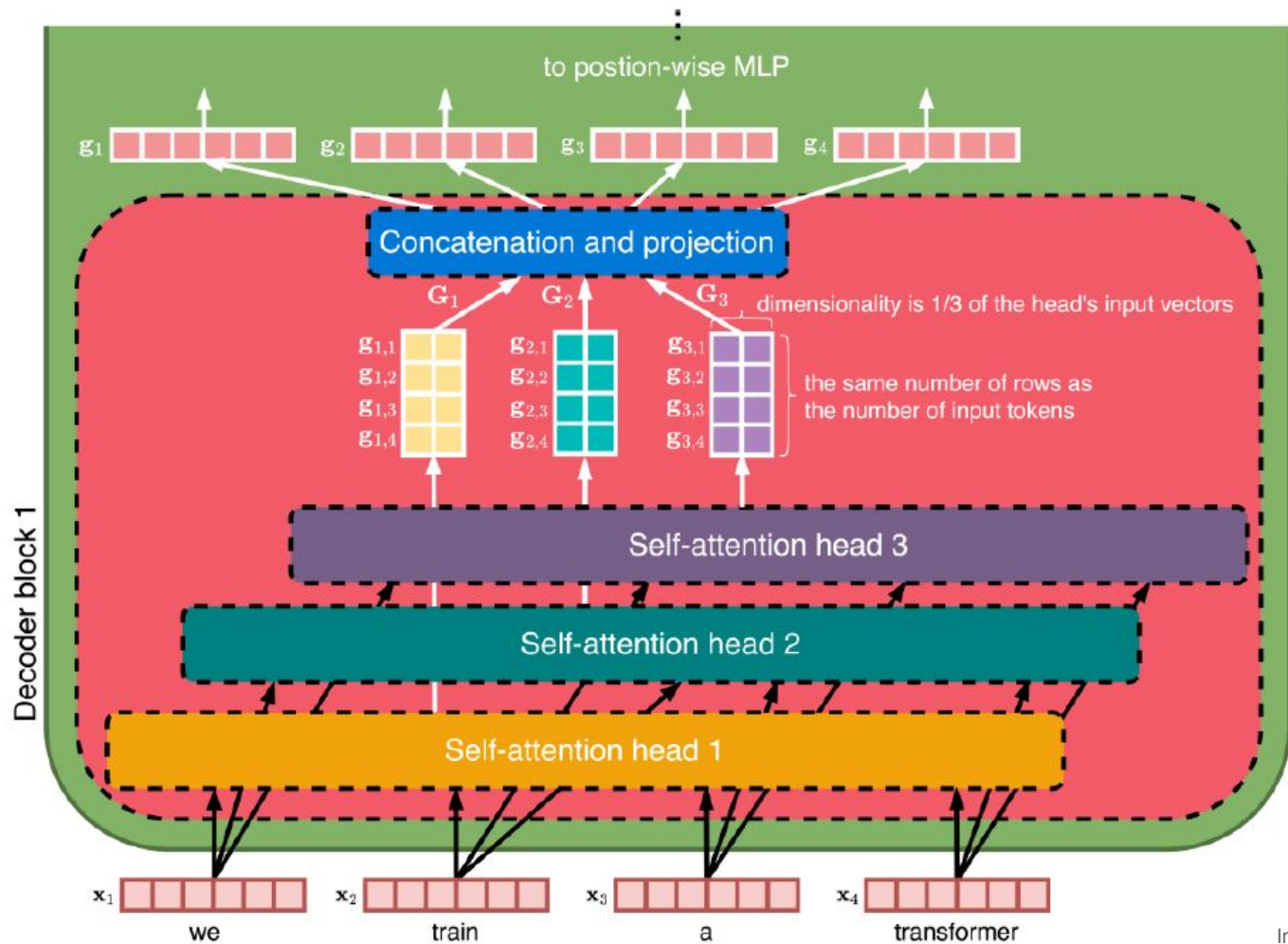
Outputs From H heads

$N \times E$ output from all heads

$d \times E$ $W_{V_{up}}$ matrices concatenated across heads

$E \times E$ output projection matrix

All of this works because number of heads chosen such that $d \times H = E$



KV caching

Intermediate values (keys and values) of all previous tokens needed during inference also, not just training

Recompute all previous keys and values for every output token, or cache keys and values of previous tokens throughput inference – compute vs memory tradeoff

	K1	K2	K3	K4
Q1	<i>prob</i>	0	0	0
Q2	<i>prob</i>	<i>prob</i>	0	0
Q3	<i>prob</i>	<i>prob</i>	<i>prob</i>	0
Q4	<i>prob</i>	<i>prob</i>	<i>prob</i>	<i>prob</i>

V1
V2
V3
V4

Attention output for 4th token is weighted sum of all prev values, weights depend on dot product of Q4 with all prev keys, so need all prev keys and values for each token

Layer Norm, Rotary Position Embedding, ...

After both attention and MLP, the output is added back to the input, and the result is normalized

Position of the token in the input is captured via position encoding (Q and K vectors are rotated by different amounts based on position)

Why? So far, we have not captured order of tokens, beyond causality. RNNs capture position by feeding tokens one by one and updating hidden state. But transformer processes all inputs at once, so positional encoding captures this information

A dropout layer “drops” some random fraction of neurons to avoid overfitting

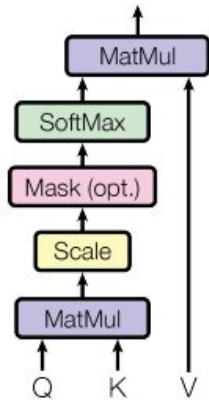
More details beyond scope of this course

“Attention is all you need”

Transformer architecture proposed in the seminal paper
“Attention is all you need” (2017, Vaswani et al)

Today’s GenAI models are mostly based on it

Scaled Dot-Product Attention



Multi-Head Attention

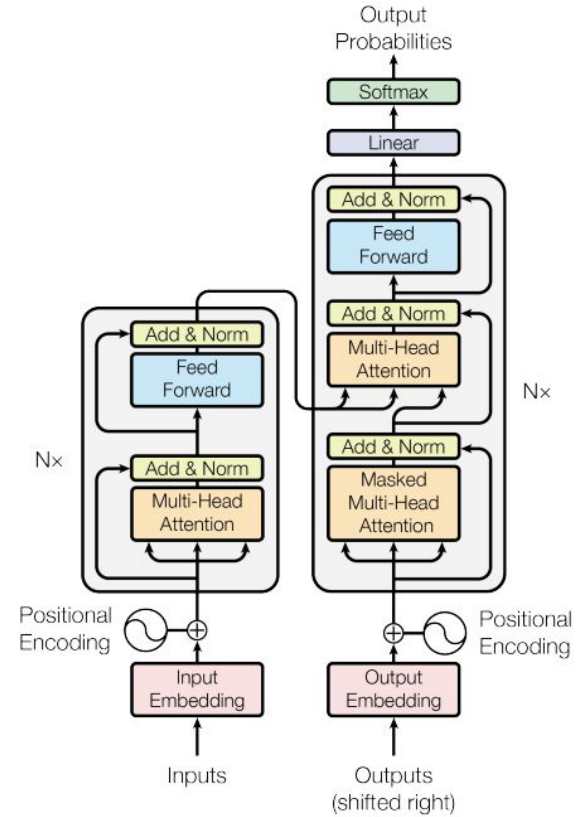
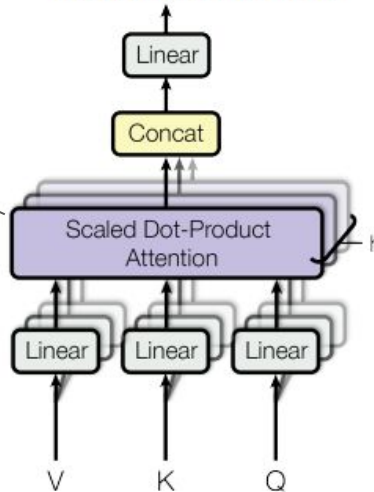


Figure 1: The Transformer - model architecture.

Example: GPT3 (175 billion parameters!)

GPT family of models are based on the transformer architecture (decoder only)

For GPT-3, we have total vocabulary size **$V = 50,257$**

Each word in vocabulary has **$E = 12,288$** dimension embedding

The model dimension **$d = 128$** , and each attention layer has **$H = 96$ heads**

Each head has 4 weight matrices for query, keys, value, and output projection

MLP layer maps embedding to hidden dimension **$d_{MLP} = 4 \times E$** , and back to E

There are **$L = 96$ layers** of such self-attention + MLP

GPT3: parameter counts

Embedding and unembedding matrices map every word in vocab to embed

Each is size $V \times E = \sim 6M$, so total $\sim 12M$

Attention layer has four weight matrices for query, key, value, projection

Each has size $d \times E \times H \times L = \sim 15B$, so total $\sim 60B$

MLP layer has two weight matrices to map E to hidden dimension and back to E

Each has size $E \times d_{MLP} \times L = \sim 60B$, so total $\sim 120B$

Total parameter count is $\sim 175B$, of which $\frac{1}{3}$ attention weights, $\frac{2}{3}$ MLP weights

We are ignoring other small things like biases, layer norm etc

Transformers: Systems implications

Other GenAI models differ in details, but attention mechanism is central to all

Large number of parameters, significant compute, memory and data movement challenges to handle all parameters

Attention requires $N \times N$ array computation, scales poorly with long input context

Memory needed to store model weights, activations and gradients for training

Memory needed to store all previous keys and values (“KV cache”) throughout inference, size increases with number of tokens

Many systems-level optimizations needed to make this all work!

Prefill vs decode

Prefill phase – process all N input tokens in parallel, compute intensive, all tokens known ahead of time, so processing for all tokens can happen simultaneously

Decode phase – generate output one token at a time. Cannot start processing for a token until previous one is ready, so cannot be done in parallel

Note that decode phase uses only previous token as input, but context of all previous tokens is embedded in the last token by virtue of attention score (which depends on all prev keys and values)

Parallelization in decode only via batching (decodes of multiple user requests in parallel)

But batching means more KV cache in memory, so no easy solutions!

Many systems issues in this course will be in the context of LLMs

Summary

We have seen a few different types of DNN architectures

Not a complete list, this is an evolving area, we have only scratched the surface

The key idea is not to know details of the ML model only, but also to understand what are the challenges in compute, memory storage, memory access patterns

We will use this knowledge to guide discussions in rest of the course (how to design software and hardware platforms, how to optimize training and inference) to meet the compute, memory, data movement requirements of ML models